

OpenClaw in the Wild: Security Analysis of Autonomous Agents

Wanlun Ma , *Member, IEEE*, Qing-Long Han , *Fellow, IEEE*, Xiaogang Zhu , *Member, IEEE*, Wei Zhou , *Member, IEEE*, Junwu Xiong , Zhihang Ren , Sheng Wen , *Senior Member, IEEE*, and Yang Xiang , *Fellow, IEEE*

Abstract—Autonomous self-hosted AI agent platforms are rapidly evolving from prompt-response assistants into persistent systems that can maintain long-lived state, invoke tools, ingest external content, and execute environment-changing actions. While this transition enables practical automation, it also introduces lifecycle security risks that cannot be fully explained by prompt-level analysis alone. In this paper, a security analysis of OpenClaw is presented, with OpenClaw serving as a representative autonomous agent operating environment and a concrete case study for broader security challenges in emerging agent ecosystems. A trust-boundary-first perspective is adopted to examine how attacks propagate across five boundary classes: Channel-Access, Session-and-State, Tool-Execution, External-Content, and Extension Supply-Chain. The results presented in this paper show that threats such as indirect prompt injection, memory poisoning, unsafe tool invocation, data exfiltration, and malicious skill abuse are not isolated anomalies; rather, they are stage-specific manifestations of a common systems problem in which untrusted influence progressively crosses into higher-privilege contexts. Based on this analysis, the defense-in-depth implications for OpenClaw deployments are discussed, including boundary-aware isolation, capability-scoped tool mediation, memory integrity controls, extension governance, and evidence-oriented operational oversight. This study provides a practical framework for evaluating and hardening long-running, tool-capable, autonomous AI agents in realistic deployment settings.

Index Terms—Agent security, Agentic AI, autonomous systems, LLM agents, tool-use safety.

I. INTRODUCTION

RECENT advances in large language models (LLMs) are rapidly transforming intelligent assistants from single-

turn conversational systems into agentic systems that can maintain memory, invoke tools, access external content, and operate persistently across heterogeneous environments [1]–[5]. This transition is significant because it changes both the capability profile and the risk profile of AI systems. Once an assistant is no longer limited to answering questions, but can instead retrieve documents, browse the web, manage files, invoke external services, and continue interaction over long-lived sessions, its behavior becomes much closer to that of a software system with delegated authority. As a result, the security analysis of agentic AI can no longer focus exclusively on model-level failures such as hallucination or jailbreaks. It should also consider how untrusted inputs, persistent state, runtime control, and environmental actuation interact within an integrated operational stack [6], [7].

Among emerging systems in this space, OpenClaw [8] is particularly noteworthy because it represents a self-hosted, gateway-centric agent operating environment rather than a lightweight orchestration wrapper. In contrast to earlier developer-facing frameworks, such as LangGraph, CrewAI, and LlamaIndex [9]–[11], OpenClaw organizes communication channels, session management, memory persistence, agent runtime, tools, skills, and extension mechanisms into a unified deployment model. Messages from external platforms are normalized through a central gateway, incorporated into persistent sessions, and processed by a long-lived runtime that can access both local and external capabilities. This architecture makes OpenClaw analytically important: it reflects a broader shift from building isolated agent pipelines to deploying fully operational assistant systems that are stateful, extensible, and directly exposed to real-world environments. In this sense, OpenClaw is not merely an implementation artifact, but a representative instance of a new class of autonomous self-hosted AI agents.

However, precisely these design choices also create a qualitatively different security landscape. In OpenClaw-like systems, security failures do not arise only from malicious prompts or unsafe generations in isolation [12]. Instead, they emerge as cross-boundary failures in which external messages, retrieved content, long-term memory, and execution privileges become tightly coupled. A malicious input may first enter through a legitimate communication channel, then be absorbed into session history, later resurface through retrieval or memory compaction, and eventually influence tool invocation or data exfiltration [13], [14]. Likewise, third-party skills

Manuscript received April 3, 2026; revised April 27, 2026; accepted May 7, 2026. Recommended by Associate Editor Derui Ding. (*Corresponding author: Sheng Wen.*)

Citation: W. Ma, Q.-L. Han, X. Zhu, W. Zhou, J. Xiong, Z. Ren, S. Wen, and Y. Xiang, “OpenClaw in the wild: Security analysis of autonomous agents,” *IEEE/CAA J. Autom. Sinica*, vol. 13, no. 6, pp. 1257–1273, Jun. 2026.

W. Ma, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang are with the School of Science, Computing and Emerging Technologies, Swinburne University of Technology, Hawthorn, VIC 3122, Australia (e-mail: wma@swin.edu.au; qhan@swin.edu.au; weizhou@swin.edu.au; swen@swin.edu.au; yxiang@swin.edu.au).

X. Zhu is with the School of Computer Science and Information Technology, Adelaide University, Adelaide, SA 5005, Australia (e-mail: xiaogang.zhu@adelaide.edu.au).

J. Xiong is with the AI Infra Team at JD Technology, Beijing 100176, China (e-mail: xiongjunwu.1@jd.com).

Z. Ren is with Google LLC, Mountain View, CA 94043, USA (e-mail: zhren@google.com).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/JAS.2026.126209

or extension packages may introduce new capabilities, new execution paths, and new privilege assumptions directly into the runtime [15], [16]. This means that the relevant security problem is no longer simply whether an LLM can resist adversarial instructions, but whether the overall system can prevent untrusted data from being elevated into privileged operational authority. For OpenClaw-like systems, the key analytical unit is therefore not the isolated prompt, but the movement of information, instructions, and permissions across trust boundaries embedded in the agent lifecycle.

This challenge is only partially addressed by the existing research. Recent work has substantially advanced the study of agent security, including prompt injection, indirect prompt injection, memory poisoning, unsafe tool use, and malicious skill ecosystems [15]–[21]. At the same time, secure-design efforts have begun to propose defense-in-depth principles for autonomous tool-using agents [13], [22]–[24]. Yet much of the current literature studies these threats in relatively isolated settings: benchmark environments, single attack surfaces, or generalized agent abstractions [12], [17], [25]. What remains insufficiently explored is a system-level, architecture-grounded analysis of self-hosted assistants in which communication ingress, session continuity, persistent memory, modular skills, and execution authority coexist within one long-lived runtime. OpenClaw makes this gap especially visible. Its gateway-centric control plane, session-centric execution model, layered memory pipeline, and extension ecosystem collectively reveal that the security of self-hosted agents is not reducible to a single failure mode; rather, it is a structured systems problem shaped by dataflow, trust boundaries, and the persistence of operational state.

In this paper, a security-oriented analysis of OpenClaw is presented as a representative autonomous self-hosted agent platform. Its architecture and workflow are first characterized, with emphasis placed on how channels, gateway logic, session state, memory, tools, and runtime coordination are integrated into a unified operating environment. The design features that make OpenClaw analytically distinctive are then examined, including gateway authority, serialized session execution, retrieval-grounded persistent memory, modular skill packaging, and embedded-yet-interoperable runtime design. To contextualize its broader relevance, the application landscape of OpenClaw-like ecosystems is further reviewed across personal assistants, multi-agent workflows, research assistants, messaging/mobile deployments, embedded assistants, and security/privacy-first systems. Building on this foundation, a dataflow-grounded and trust-boundary-oriented taxonomy of OpenClaw security risks is developed, covering channel-access, session-and-state, tool-execution, external-content, and extension supply-chain boundaries, together with their cross-boundary propagation patterns. Finally, mitigation and defense-in-depth strategies are discussed, and open challenges for making such systems more secure, observable, and deployable in practice are outlined.

To explicitly define the scope of this work, it should be noted that this paper is structured as an architecture-grounded security analysis and system-level case study, rather than an exhaustive catalog of autonomous-agent research. OpenClaw

is utilized as a representative autonomous agent operating environment through which broader system-level risks can be concretized. We utilize OpenClaw as a concrete analytical vehicle to investigate a specific, under-explored domain: *how cross-boundary attack propagation manifests within persistent, self-hosted autonomous agent runtimes*. Throughout this paper, we use autonomous agents as the primary term for long-running, tool-capable LLM systems. The expression agentic AI is retained only when referring to the broader literature that adopts that wording.

The main contributions of this paper are as follows:

- 1) OpenClaw is positioned as an autonomous agent operating environment, and its distinction from prior orchestration frameworks that primarily function as development libraries is clarified.
- 2) The architectural and design features that are most security-relevant in OpenClaw are identified, including its gateway-centric control plane, persistent session semantics, layered memory subsystem, and extensible skill/runtime ecosystem.
- 3) A dataflow-grounded trust-boundary taxonomy is proposed to explain how diverse threats, including prompt injection, memory poisoning, unsafe execution, malicious external content, and extension compromise, can be understood within a unified analytical framework.
- 4) Current mitigation ideas are synthesized into a defense-in-depth perspective, and open research directions for building security-assurable self-hosted AI agents are highlighted.

The remainder of this paper is organized as follows. Section II presents the architecture and end-to-end workflow of OpenClaw. Section III analyzes the design features that shape its security posture. Section IV reviews representative applications of OpenClaw-like ecosystems to contextualize deployment assumptions and risk exposure. Section V introduces a trust-boundary-oriented threat taxonomy and examines boundary-specific and cross-boundary attack propagation. Section VI discusses mitigation and defense-in-depth strategies, and Section VII outlines open challenges and future research directions. Finally, Section VIII concludes the paper.

II. OPENCLAW ARCHITECTURE AND WORKFLOW

A. Architecture Overview

Fig. 1 presents the overall architecture of OpenClaw as a gateway-centered orchestration system composed of seven tightly connected parts: Channels, Gateway, Sessions and Memory, Agent Runtime, Model Providers, Tools and Skills, and Extensions. Rather than treating agent execution as an isolated prompt–response process, OpenClaw organizes the entire lifecycle of interaction into a persistent operational framework. User messages first arrive through external channels and are normalized into a common message representation. These messages are then handled by the Gateway, which performs routing, session coordination, policy mediation, and control-plane management. The Agent Runtime operates on top of this orchestration layer by assembling context, invoking models, interacting with tools and workspace resources, and generating responses. At the same time, session tran-

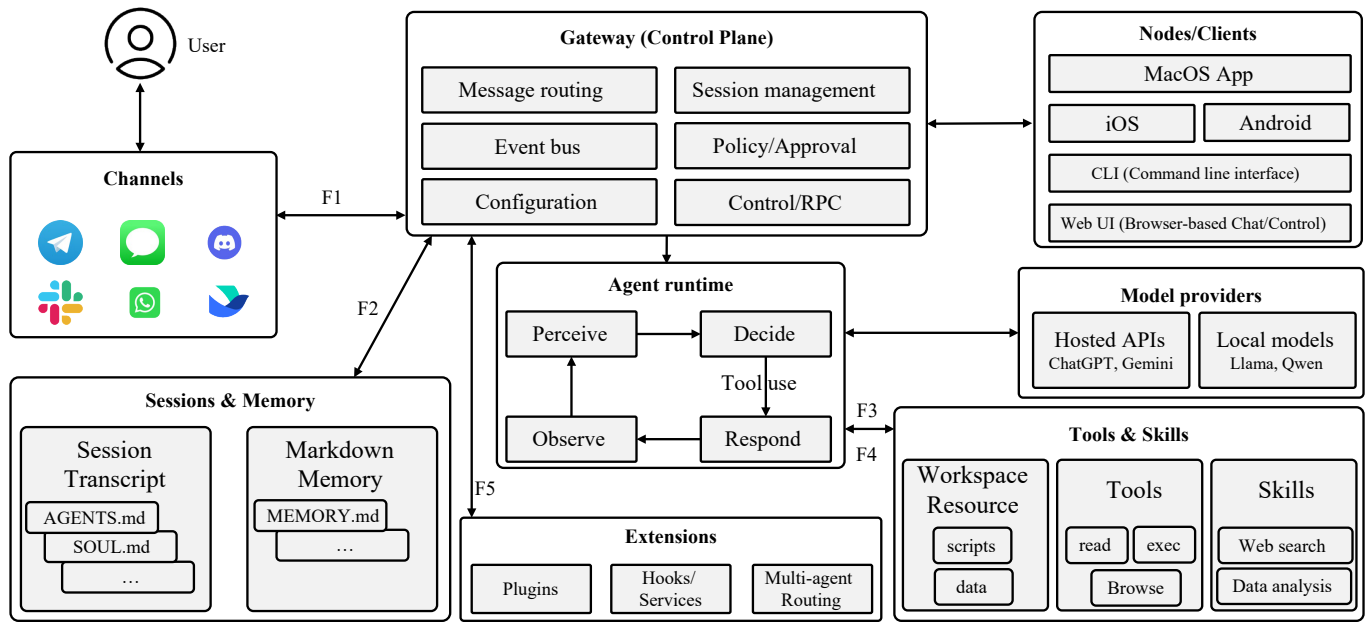


Fig. 1. System architecture diagram of OpenClaw. The labels F_1 – F_5 correspond to the Channel-Access (F_1), Session-and-State (F_2), Tool-Execution (F_3), External-Content (F_4), and Extension Supply-Chain (F_5) boundaries analyzed in Section V.

scripts and Markdown-based memory provide persistent context, while extensions, plugins, and multi-agent routing mechanisms act as a lower-layer substrate that broadens the system's capabilities. This architecture makes OpenClaw not simply a model wrapper, but a structured agent system that integrates communication, orchestration, execution, persistence, and extensibility into a unified workflow.

B. Core Components and End-to-End Workflow

OpenClaw organizes agent execution as a coordinated system rather than a single prompt-response interface. Its workflow begins with a *channel* layer that receives inbound and outbound communications from external platforms such as WhatsApp, Telegram, Discord, Slack, and iMessage. Instead of treating each channel as a separate execution path, OpenClaw normalizes heterogeneous message inputs into a shared internal representation. This design allows user interactions originating from different communication surfaces to enter the same orchestration pipeline and be processed through a unified control model.

The central coordinating component of this pipeline is the *Gateway* layer. In the proposed architecture, the Gateway integrates message routing, session management, event handling, policy and approval logic, configuration, control, and remote procedure call (RPC) functions into one long-lived operational core. Through this design, OpenClaw separates transient user interfaces from persistent system coordination. Channels, clients, and other connected components do not directly manage their own isolated execution state; instead, they interact through a common orchestration layer that governs how messages are routed, how sessions are maintained, and how runtime execution is coordinated across the system.

To support stateful interaction, OpenClaw introduces the state management layer for *sessions*, *memory*, and *context construction*. This layer maintains Session Transcripts to pre-

serve evolving conversational history, and Markdown Memory to externalize durable, editable records such as user profiles and long-term system state. Instead of relying exclusively on the transient context window of a large language model, OpenClaw utilizes a Context Assembly mechanism to systematically synthesize these persistent files into a structured working context. This design explicitly decouples persistent state preservation from immediate model inference, treating historical logs and explicit file-based memory as first-class operational drivers.

The actual execution pipeline is anchored by the *Agent Runtime*, depicted as an iterative loop of Perceive, Decide, Respond, and Observe. This cyclical topology underscores that runtime execution is inherently interactive. The runtime evaluates the assembled context, determines sequential actions, and observes environmental state changes before reiterating the loop. Concurrently, external *Models and API Providers* supply the underlying cognitive inference. Thus, the runtime acts as the operational engine translating orchestrated inputs into embodied behavior, while the externalized models function purely as decoupled inference engines.

Finally, serving as the foundational substrate across the entire architecture, the *Extensions and Multi-Agent Routing* layer facilitates system-wide capability augmentation. This cross-cutting layer incorporates plugins, custom hooks, context engines, and routing protocols, enabling the delegation of complex workloads across specialized agent topologies. Conceptually, this substrate confirms OpenClaw's design as a highly extensible ecosystem. However, it also signifies that the system's operational boundaries are fundamentally porous, continuously ingesting third-party logic and unverified extensions into the core execution loop.

C. OpenClaw in Relation to Prior Agent Frameworks

To contextualize OpenClaw within the broader ecosystem, it

is critical to delineate its level of abstraction relative to prior agent frameworks [26], such as LangGraph [9], CrewAI [10], and LlamaIndex [11]. These established solutions function primarily as developer-facing orchestration libraries designed to construct agentic applications. Specifically, LangGraph provides low-level infrastructure for stateful, long-running workflows with an emphasis on durable execution. CrewAI structures multi-agent collaboration around event-driven flows and specialized collaborative agentic AI “crew”. Similarly, LlamaIndex offers a flexible architecture for routing and event-driven workflows. Fundamentally, these paradigms serve as middleware or application-layer building blocks that require external host-environment integration.

From a security-architecture perspective, this distinction is important because the locus of security enforcement differs across these systems. In frameworks such as LangGraph, CrewAI, and LlamaIndex, security-relevant mechanisms are generally exposed as workflow-, tool-, or application-level controls that developers compose into a larger host environment. For example, LangGraph emphasizes durable execution, checkpointed state, interruptible workflows, and human-in-the-loop control; CrewAI provides agent, crew, flow, tool, guardrail, memory, and observability abstractions for constructing collaborative agent systems; and LlamaIndex focuses on data-grounded agents, retrieval pipelines, tool use, and developer-controlled agent workflows. These mechanisms can support secure application construction, but they do not by themselves define a unified security perimeter for multi-channel ingress, session admission, persistent memory, local workspace access, tool execution, and third-party extension loading. In OpenClaw, by contrast, these concerns are integrated into a self-hosted operating environment centered on the Gateway and runtime. The security boundary is therefore not only the agent workflow designed by a developer, but the full platform boundary through which external messages, long-lived state, tools, local resources, and extensions are admitted, mediated, and persisted. This makes OpenClaw’s security architecture closer to that of an exposed autonomous runtime than a library-level orchestration framework.

This difference becomes clearer in OpenClaw’s platform architecture. Rather than serving merely as a modular dependency, it is engineered as a standalone platform where a singular Gateway process acts as the definitive source of truth for sessions, routing, and channel connections. Beyond multi-channel message ingress, its operational stack natively amalgamates persistent session management, workspace-aware execution, multi-agent routing, a browser-based control UI, and remote mobile nodes. Consequently, OpenClaw should not be interpreted as a direct substitute for earlier orchestration libraries. Instead, it occupies a distinct, high-level system-integration tier where communication interfaces, orchestration logic, state persistence, and environmental actuation are fused into a unified runtime. However, it is precisely this architectural paradigm shift—collapsing the traditional isolation boundaries between untrusted external ingress and local system actuation—that fundamentally alters the threat landscape, transforming its integration strengths into critical security vulnerabilities.

III. DISTINCTIVE DESIGN FEATURES OF OPENCLAW

Building on the architectural overview in Section II, this section examines the design choices that make OpenClaw analytically distinctive as a self-hosted agent system. The purpose here is not to restate the end-to-end workflow, but to explain why several core mechanisms in OpenClaw deserve attention as system-level design features in their own right. In particular, OpenClaw is organized around a long-lived gateway, persistent session state, explicit memory files, modular capability packaging through skills, and an embedded runtime strategy that is further extensible to external agent backends. Taken together, these mechanisms indicate that OpenClaw should be understood not merely as a thin orchestration layer for model calls, but as a coordinated agent operating environment that integrates communication channels, runtime control, memory persistence, and execution affordances into a single system architecture.

A. Gateway-Centric Control Plane

The foundational architectural differentiator of OpenClaw is its gateway-centric control plane. Rather than positioning the gateway at the periphery as a passive transport adapter, OpenClaw positions it as the single source of truth for sessions, routing, and channel connections. In practical terms, this means that inbound communication, session resolution, reply routing, and channel coordination are all anchored in a persistent gateway process. Such an arrangement is significant because it defines the system around continuous coordination and state management, rather than around isolated request-response interactions with a language model.

This gateway-centric design has important architectural consequences. Because the gateway owns session and routing authority, multiple channels and interfaces can be connected to a unified stateful backend without requiring each client to maintain a separate, fully authoritative conversation history. In this architecture, gateway-backed transcripts, rather than individual front-end clients, function as the canonical view of session state. This organization supports multi-channel usage while also simplifying the enforcement of access control, pairing, and routing decisions at a central point. It therefore turns the gateway into a control layer that coordinates communication surfaces, agent contexts, and execution state in a coherent manner.

The same design also clarifies why OpenClaw should be analyzed as a system rather than as a prompt wrapper. A system in which the gateway determines session continuity, channel admission, and routing behavior necessarily embeds policy and coordination logic into the platform itself. In other words, OpenClaw does not merely run an agent behind a chat interface; it uses the gateway to organize how stateful agent behavior is exposed, synchronized, and controlled across different interaction surfaces. This feature provides the conceptual basis for the remaining design choices discussed below, especially those involving session semantics, memory persistence, and serialized execution.

B. Session-Centric Runtime and Serialized Execution

Building upon this centralized control, OpenClaw elevates

the session—rather than the individual model inference—to the primary unit of runtime organization. The system maintains explicit session keys, session identifiers, transcript persistence, and configurable scoping policies such as `dmScope` for controlling how direct-message conversations are isolated or shared. This indicates that a session in OpenClaw is not simply an ephemeral context window assembled for one interaction, but a durable execution context whose lifecycle is managed by the system over time.

The runtime design reinforces this session-first orientation through serialized execution. Within OpenClaw’s agent loop, a run encompasses the full authoritative path from intake and context assembly to model inference, tool execution, streaming replies, and persistence. Crucially, this loop is serialized per session, with `runEmbeddedPiAgent` coordinating execution through per-session and optional global queues. This mechanism is designed to prevent session and tool races while preserving the consistency of session history. This choice is more than an implementation detail: it embodies a particular runtime semantics in which the integrity of the session transcript and the consistency of tool side effects are treated as system invariants.

This session-centric model is also important from a security and privacy perspective. OpenClaw warns that when multiple users share the same Direct Message (DM) session, private context can leak across senders, and it therefore recommends enabling secure DM mode in multi-user settings. The need for such guidance reveals that session scoping is not a cosmetic configuration option but a core boundary-setting mechanism for context isolation. Put differently, OpenClaw’s runtime behavior is inseparable from how sessions are defined, partitioned, and serialized. This makes session design a first-class analytical feature of the platform and a natural bridge to later discussions of trust boundaries and attack surfaces.

C. Layered Memory, Retrieval, and Persistent Compaction

To sustain long-lived interactions, OpenClaw implements a sophisticated memory pipeline that couples file-based persistence with structured indexing and retrieval-time ranking. While Markdown files (e.g., `MEMORY.md`) serve as the definitive source of truth, the system abstracts this storage behind a configurable embedding and indexing layer. Architecturally, OpenClaw supports local models alongside remote providers via the `agents.defaults.memorySearch` configuration, dynamically routing requests based on available credentials and deployment modes.

The retrieval mechanism employs a hybrid search architecture, fusing semantic vector similarity with exact lexical matching (BM25) [27]. This ensures that while vector search captures semantic intent, BM25 retains high precision for exact tokens like environment variables and code symbols. The system normalizes and unions candidate sets, computing a weighted final score:

$$FinalScore = W_{vec} \times Score_{vec} + W_{text} \times Score_{text}$$

By default, semantic similarity is prioritized ($W_{vec} = 0.7$), yet the system degrades gracefully to BM25-only or vector-only modes if either backend fails.

Following retrieval, OpenClaw applies temporal decay and Maximal Marginal Relevance (MMR) re-ranking. Temporal decay imposes an exponential recency boost to prioritize newer context:

$$decayedScore = score \times e^{-\lambda \times ageInDays}$$

Crucially, evergreen files are exempt from this decay, acting as stable knowledge anchors. MMR is subsequently applied to penalize near-duplicate snippets, explicitly balancing relevance against informational diversity.

At the data layer, OpenClaw leverages the `sqlite-vec` extension to perform vector distance computations directly within a SQLite `vec0` virtual table, avoiding the overhead of loading embeddings into application memory. Chunk embeddings are aggressively cached to optimize incremental indexing. Furthermore, the architecture supports an experimental local-first sidecar, which decouples the retrieval engine from the persistence surface while maintaining a unified agent-facing interface.

Finally, retrieval is tightly coupled with persistence through pre-compaction memory transfer. Before a session hits its context boundary, the system executes a silent turn (suppressed via a `NO_REPLY` convention), forcing the agent to flush volatile state into durable workspace files. This mechanism proves that OpenClaw does not merely truncate history; it proactively converts conversational state into retrievable knowledge, establishing a highly integrated memory subsystem rather than a naive append-only log.

D. Skills as Modular Capability Packaging

If the memory subsystem provides durable state, the Skills architecture enforces a disciplined, technical mechanism for operational capability packaging. Centered around structured `SKILL.md` artifacts, skills function as an intermediate abstraction between the language model and the host environment. They encode frontmatter, operational constraints, and tool-use affordances, governing exactly when and how an agent interacts with system resources.

Skill resolution operates through a strict hierarchical precedence model. Capabilities are aggregated across bundled defaults, managed directories, and workspace-specific paths (`<workspace>/skills`), with workspace-local skills overriding global ones. This compositional layering permits granular capability specialization across multi-agent deployments. Furthermore, skill eligibility is treated as a deterministic technical filtering problem rather than a semantic prompt decision. Through metadata declarations, each skill enforces prerequisites such as operating-system constraints, required binaries, and specific environment variables (e.g., `primaryEnv`). At load time, OpenClaw evaluates these gates, ensuring the model is never exposed to capabilities it cannot physically execute within its current sandbox or host environment.

A particularly distinctive engineering choice is OpenClaw’s approach to context-window efficiency via progressive disclosure. Rather than indiscriminately injecting full skill instructions into the system prompt, the embedded runtime synthesizes a compact XML inventory of eligible capabilities via the `formatSkillsForPrompt` routine. This inventory is explicitly

optimized for token efficiency: a base overhead of 195 characters, plus approximately 97 characters per skill for routing metadata. Consequently, the model first processes a low-footprint capability index, only dynamically fetching granular instructions when a specific skill is invoked.

Operationally, skills are tightly bound to runtime state management. Upon initiation, OpenClaw snapshots eligible skills and temporarily injects required secrets (via `skills.entries.<key>.apiKey`) into the process environment exclusively for the duration of the agent's turn. This strict scoping prevents credential leakage into persistent global shell states. The system also supports hot-reloading through debounced watchers, allowing dynamic capability updates without session restarts.

Ultimately, this modular flexibility fundamentally expands the system's attack surface. Because third-party skills (e.g., those synchronized via the ClawHub registry [28]) dictate tool invocation and environment exposure, they inherently function as untrusted executable code. The dual nature of the Skills subsystem—providing sophisticated, token-efficient capability orchestration while simultaneously introducing complex supply-chain and privilege-escalation risks—makes it a critical focal point for security analysis.

E. Embedded Pi Runtime and External Agent Interoperability

Finally, the execution of these capabilities is governed by a deeply embedded runtime strategy. The significance of Pi integration is not simply that OpenClaw adopts another agent technology, but that it embeds the Pi SDK into its own gateway- and session-centered execution model [29]. Rather than spawning Pi as a subprocess or accessing it through an external RPC layer, OpenClaw directly instantiates Pi's `AgentSession` through `createAgentSession()` [29]. The resulting design affords full control over session lifecycle and event handling, custom tool injection, system prompt customization, session persistence, authentication profile rotation, and provider-agnostic model switching. These details indicate that Pi is incorporated as a runtime substrate within OpenClaw's own system semantics, rather than being attached as an externally managed component.

This embedded design is tightly coupled to the authoritative agent loop. Within this loop, the agent command resolves model settings, loads a skills snapshot, invokes `runEmbeddedPiAgent`, and bridges Pi-generated assistant, tool, and lifecycle events back into OpenClaw's own event stream. This means that OpenClaw preserves architectural ownership over session preparation, prompt construction, stream handling, timeout control, and persistence even while delegating core agent execution to the embedded Pi runtime. In analytical terms, the system does not collapse into Pi; instead, Pi is subordinated to OpenClaw's broader control plane and lifecycle logic.

OpenClaw also extends beyond a single embedded runtime through explicit support for external agent interoperability. Through an ACP backend plugin, OpenClaw can run external coding harnesses such as Pi, Claude Code, Codex, OpenCode, and Gemini CLI, and natural-language requests can be routed to this ACP runtime when appropriate [30]. This reveals an

important strategic choice: OpenClaw is not designed around exclusive dependence on one native runtime. Rather, it combines an embedded in-process runtime with an interoperability layer for external agent backends. Such a design enhances flexibility while preserving a consistent gateway- and session-managed interaction surface.

Taken together, these runtime choices reinforce the argument of this section. OpenClaw's distinctiveness does not depend on any single mechanism being entirely unprecedented in isolation. Instead, its analytical value lies in how gateway authority, session-centric execution, file-grounded memory, modular skills, and embedded-yet-interoperable runtime design are combined into a coherent self-hosted agent system. This combination gives OpenClaw a system profile that is especially relevant for architectural analysis and, subsequently, for security-oriented study.

IV. APPLICATIONS OF OPENCLAW-LIKE ECOSYSTEMS

This section reviews the application landscape of OpenClaw-like ecosystems. Table I summarizes these application categories and representative examples. Rather than forming a single class of general-purpose assistants, these systems have diversified into a broad range of practical settings, including full-featured personal assistants, multi-agent workflow platforms, privacy- and security-oriented systems, messaging and mobile assistants, research-oriented tools, and embedded or edge assistants for resource-constrained environments. Such diversity indicates that recent agent ecosystems are increasingly shaped by concrete deployment needs, where differences in autonomy, interaction modality, privacy requirements, computational resources, and orchestration complexity give rise to distinct application categories. Examining these applications therefore helps clarify how OpenClaw-like ecosystems are evolving toward more specialized and context-aware forms of assistant design.

A. General-Purpose Assistants

General-purpose personal assistant platforms represent one of the most prominent application categories within OpenClaw-like ecosystems. Unlike narrowly specialized agents designed for a single interface or task, these systems aim to provide a broad assistant experience by integrating multiple capabilities, such as task routing, conversational interaction, voice support, and access across different communication channels. Their design typically emphasizes flexibility and extensibility, allowing the assistant to operate as a unified entry point for diverse user requests rather than as a standalone tool for a fixed workflow. For example, OpenClaw [8] is positioned as a personal AI assistant with broad functionality, reflecting an effort to support rich interaction and general-purpose assistance in a self-hosted setting. Similarly, Autobot [31] highlights an ultra-efficient personal assistant design, suggesting that general-purpose capability can also be pursued under stronger efficiency constraints. Pickle-Bot [32] is explicitly presented as a personal AI assistant, reflecting the continuing demand for simple, user-facing assistants that prioritize accessibility and convenience. Taken together, these platforms illustrate how OpenClaw-like ecosystems are not

TABLE I
APPLICATIONS OF OPENCLAW-LIKE ECOSYSTEMS

Category	Description	Representative Examples
General-purpose assistants	Broad assistants with multi-channel interaction, memory, tools, and often voice or vision.	OpenClaw [8], Autobot [31], Pickle-Bot [32]
Multi-agent/workflow systems	Systems centered on chained execution, task coordination, or large-scale orchestration across multiple agents.	Moxxy [33], TinyClaw [34]
Research assistants	Assistants for note generation, report writing, scripting, experimental exploration, or research-oriented workflows.	HermitClaw [35], Nanobot [36], OpenFang [37]
Messaging/mobile automation assistants	Assistants tailored for messaging platforms, mobile environments, or broad multi-platform communication channels.	AstrBot [38], ZeroClaw [39], SupaClaw [40], LettaBot [41], Flowly [42], DroidClaw [43], BabyClaw [44]
Embedded/edge assistants	Assistants targeting constrained devices such as older phones, low-power hardware, or embedded environments.	Zclaw [45], MimiClaw [46], Picobot [47], PicoClaw [48]
Security/privacy-first assistants	Assistants that prioritize sandboxing, encrypted state, isolation, secret scanning, or reduced reliance on remote models.	AngelClaw [49], SafeClaw [50], Shrew [51], IronClaw [52], Moltis [53]

limited to experimental agent frameworks, but are increasingly evolving toward practical, full-featured assistant systems that seek to balance versatility, usability, and deployability.

B. Multi-Agent/Workflow Systems

Multi-agent and workflow orchestration systems form another important application category within OpenClaw-like ecosystems. In contrast to general-purpose personal assistants that primarily center on direct user interaction, this class emphasizes the coordination of multiple agents, chained task execution, and tool-driven workflows to support more structured and scalable forms of automation. Such systems are typically designed to decompose complex objectives into smaller subtasks, assign responsibilities across specialized agents or modules, and manage the flow of intermediate outputs throughout a larger pipeline. This application orientation is particularly suitable for scenarios involving repeated operational routines, collaborative task structures, or semi-autonomous execution beyond a single conversational turn. For example, Moxxy [33] is positioned around everyday AI assistant agents, reflecting an emphasis on agent-based delegation in routine user workflows. Similarly, TinyClaw [34] explicitly presents itself as an agent-teams orchestrator, highlighting the role of coordinated multi-agent execution for individual users or small-scale organizations. Together, these systems illustrate how OpenClaw-like ecosystems are expanding from single-assistant paradigms toward orchestrated agent collectives, where the primary value lies not only in responding to user queries, but also in organizing, sequencing, and executing interconnected tasks in a more autonomous and workflow-aware manner.

C. Research Assistants

Research assistant applications represent a more exploratory and productivity-oriented category within OpenClaw-like ecosystems. Rather than primarily targeting general conversational assistance or large-scale workflow coordination, these systems are often oriented toward supporting report drafting, note generation, script-based interaction, and experimental use in personal or research settings. Their design typically emphasizes lightweight deployment, close integration with local working environments, and the ability to assist with iterative

knowledge work in a flexible manner. For example, HermitClaw [35] is described as a tiny AI creature that lives in a folder on a user's computer, reflecting a highly localized and personal form of assistant design that is well suited to ongoing note-taking, lightweight writing support, and experimental interaction. Nanobot [36], as an ultra-lightweight personal AI assistant, similarly suggests a minimal yet practical assistant model that can support everyday research-related tasks without the overhead of a large platform. By contrast, OpenFang [37], positioned as an agent operating system, indicates that this category can also extend toward more structured experimental environments, where the assistant is not only a tool for producing text or notes, but also a substrate for testing agentic behaviors and coordinating research-oriented functions. Taken together, these systems highlight an important direction in OpenClaw-like ecosystems, namely the development of assistants that are closely aligned with individual knowledge work, local experimentation, and lightweight research productivity, bridging the gap between personal assistance and agentic support for exploratory computational tasks.

D. Messaging/Mobile Automation Assistants

Messaging, mobile, and cross-platform assistants represent an application category in which OpenClaw-like ecosystems are shaped primarily by communication channels and device-level accessibility rather than by a single standalone interface. In contrast to assistants that operate mainly within a dedicated desktop or web environment, these systems are designed to integrate with instant messaging platforms, mobile devices, and heterogeneous chat channels, thereby enabling users to interact with agentic capabilities in more pervasive and contextually embedded ways. This orientation is particularly important for real-world adoption, as it reduces friction between assistant functionality and the communication tools that users already rely on in daily life. For example, AstrBot [38] explicitly targets agentic chatbot infrastructure for instant messaging environments, highlighting the role of native deployment in assistant design. Flowly [42] further emphasizes cross-channel accessibility through its positioning as a local AI that "talks everywhere", suggesting a model in which locality of execution is combined with broad communication reach. DroidClaw [43] extends the category toward mobile-

device interaction by focusing on Android phone control, illustrating how OpenClaw-like ecosystems can move beyond chat assistance into direct mobile automation. Similarly, BabyClaw [44] demonstrates a Telegram-centered deployment pattern, where a remotely hosted agent is controlled through a widely used messaging application. Although systems such as ZeroClaw [39], SupaClaw [40], and LettaBot [41] are not limited to a single messaging platform, their positioning still aligns with this category in that they emphasize personal assistant functionality delivered through accessible, distributed, or memory-rich interaction channels. Overall, these examples show that OpenClaw-like ecosystems are increasingly adapting to messaging-centric, mobile, and cross-platform settings, reflecting a broader shift toward assistants that are not only intelligent, but also ubiquitously reachable across the interfaces and devices that structure everyday digital communication.

E. Embedded/Edge Assistants

Embedded, edge, and resource-constrained assistants represent a distinctive application category within OpenClaw-like ecosystems, where the primary design challenge is to deliver useful assistant functionality under strict limitations in compute, memory, energy, or device form factor. In contrast to cloud-oriented or desktop-scale agent systems, these assistants are intended for deployment on low-power hardware, older mobile devices, compact local platforms, or highly lightweight runtime environments. As a result, their development is typically shaped by strong efficiency requirements, including small system footprints, simplified execution pipelines, reduced dependency on heavyweight infrastructure, and tighter integration with local hardware constraints. This category is important because it extends the reach of assistant ecosystems beyond well-provisioned computing settings, enabling more portable, affordable, and ubiquitous forms of deployment. For example, MimiClaw [46] described as a pocket AI assistant on a low-cost chip, directly reflects the ambition of bringing assistant capabilities to minimal hardware platforms. PicoClaw [48] similarly emphasizes ultra-efficient implementation, suggesting that careful systems design can make assistant functionality viable even in constrained environments. Picobot [47], positioned as an agent that runs anywhere, highlights portability as a central goal, while Zclaw [45] further illustrates the tendency of this category to prioritize compactness and deployability over large-scale functionality. Taken together, these systems show that OpenClaw-like ecosystems are not only moving toward greater autonomy and specialization, but are also being adapted to settings where efficiency and hardware accessibility are first-order concerns, thereby broadening the practical scope of agentic assistants across edge and embedded computing scenarios.

F. Security/Privacy-First Assistants

Privacy-, safety-, and security-first assistant systems constitute an important application category within OpenClaw-like ecosystems, reflecting a growing recognition that assistant functionality must be accompanied by stronger guarantees

regarding trustworthiness, isolation, and operational control. Unlike general-purpose assistants that primarily optimize for versatility or convenience, this class emphasizes mechanisms such as encrypted or protected state management, sandboxed execution, tenant isolation, secret-aware operation, and, in some cases, reduced dependence on heavyweight remote LLM services. Such design choices are particularly relevant as agentic assistants gain access to sensitive user data, local files, credentials, and multi-step tool execution, all of which increase the potential impact of failures or misuse. For example, AngelClaw [49], described as a multi-tenant agent operating system, points to a setting in which isolation and controlled sharing of resources become central concerns. SafeClaw [50] further suggests an effort to provide a safer and more accessible alternative within this ecosystem, while Shrew [51] highlights how lightweight design can also support a smaller and potentially more controllable execution surface. IronClaw [52] most directly reflects the secure-first orientation through its explicit positioning as a secure personal AI assistant, indicating that security is treated not as an auxiliary feature but as a primary design objective. Similarly, Moltis [53], presented as a Rust-native claw “you can trust”, reinforces the connection between implementation choices and trust-oriented system design, where language-level safety and reliability may play an important role. Taken together, these systems illustrate that OpenClaw-like ecosystems are increasingly evolving beyond raw assistant capability toward more security-conscious forms of deployment, in which privacy preservation, execution safety, and trustworthy operation become core requirements rather than optional enhancements.

V. SECURITY THREATS TO OPENCLAW

A. Risk Taxonomy

Building upon the architectural foundation introduced in Section II, the security vulnerabilities of OpenClaw are viewed not as isolated component failures, but as systemic breakdowns in data and capability flows. Because OpenClaw tightly couples multi-channel ingress, centralized orchestration, persistent memory, and extensible execution within a unified environment, security-relevant behaviors emerge dynamically as information traverses the system. Consequently, the attack surface is fundamentally dataflow-driven: vulnerabilities manifest when untrusted inputs are progressively transformed into persistent state, tool invocations, or externally visible actuation.

To formalize this threat landscape, we structure our taxonomy around five critical operational data flows (F_1 to F_5). Rather than merely describing chronological execution, these flows delineate the exact junctures where untrusted input, delegated authority, persistent context, and executable extensions intersect:

- 1) F_1 (**Channel** $\rightarrow$ **Gateway**): The initial ingress of external messages into the orchestration core.
- 2) F_2 (**Gateway** $\rightarrow$ **Agent**): The assimilation of routed messages into durable session state and memory.
- 3) F_3 (**Agent** $\rightarrow$ **Tools**): The translation of cognitive deci-

TABLE II
A DATAFLOW-GROUNDED TRUST-BOUNDARY TAXONOMY OF OPENCLAW SECURITY RISKS

Operational Data Flow	Trust Boundary	Security Focus	Representative Risks	Primary Consequence
F_1 : Channel $\rightarrow$ Gateway	Channel-Access Boundary	Authentication and authorization of inbound traffic to the orchestration core	Pairing-code interception; allowFrom spoofing; token theft; exposed websocket surfaces; direct prompt injection	Unauthorized influence enters the orchestration pipeline, establishing a beachhead for downstream attacks.
F_2 : Gateway $\rightarrow$ Agent	Session-and-State Boundary	Integrity of context windows, transcript logs, and file-grounded memory	Cross-session contamination; transcript leakage; persistent memory poisoning; context drift; configuration tampering	Short-lived attacker influence becomes persistent operational leverage that survives beyond the initial interaction.
F_3 : Agent $\rightarrow$ Tools	Tool-Execution Boundary	Containment and authorization of physical/system-level actuation commands	Tool argument injection; execution approval bypass; unauthorized shell commands; resource exhaustion	Adversarial influence is converted into tangible manipulation of local software, files, or networked resources.
F_4 : Agent $\rightarrow$ External	External-Content Boundary	Sanitization of asynchronously retrieved web and document data	Indirect prompt injection; retrieval poisoning; tool-result injection; hidden instruction carriers (web_fetch)	Untrusted external content is reinterpreted as control-bearing input, hijacking subsequent planning or execution.
F_5 : ClawHub $\rightarrow$ Agent	Extension Supply-Chain Boundary	Verification of dynamically loaded capability profiles	Malicious skill installation; poisoned updates; hidden instruction payloads; credential harvesting; capability escalation	External trust failure embeds durable, malicious executable capability directly inside the runtime environment.

sions into concrete, system-level tool invocations.

4) F_4 (**Agent** $\rightarrow$ **External**): The retrieval and ingestion of unverified external web or document content.

5) F_5 (**ClawHub** $\rightarrow$ **Agent**): The integration of third-party capability packages into the native runtime.

Building on these data flows, the proposed taxonomy is architecture-guided and dataflow-guided. It is derived from OpenClaw's actual control structure and operational data flows, as characterized in Sections II and III, rather than from an abstract enumeration of generic agent threats. In this sense, the five categories correspond to the major trust boundaries exposed by how messages, state, retrieved content, tool authority, and third-party capabilities move across the system lifecycle. These categories are analytically separable, although a real-world attack may span multiple boundaries over time. Accordingly, we classify threats by the primary boundary crossed at a given stage, while downstream cross-boundary propagation is discussed separately in Section V-G.

As data traverses these flow paths, it crosses distinct trust boundaries, each dictating a specific class of security assumption. Accordingly, OpenClaw's attack surface can be categorized into five boundary-oriented risk domains: *Channel-Access*, *Session-and-State*, *Tool-Execution*, *External-Content*, and *Extension Supply-Chain* boundaries. This taxonomy isolates where security invariants are violated: who dictates system entry (F_1), how transient input poisons durable memory (F_2), when model hallucinations become physical execution (F_3), how external data hijacks control flow (F_4), and how third-party code bypasses authorization (F_5).

Under this paradigm, phenomena like prompt injection, unauthorized execution, and malicious skill installation are not disparate anomalies, but boundary-specific manifestations of a common system-level problem: the elevation of unverified data into privileged execution contexts. Table II summarizes this taxonomy, providing the analytical scaffold for the detailed vulnerability analysis in the subsequent subsections. (Note that memory is treated herein as an integral component of the Session-and-State boundary, as OpenClaw materializes

persistent context holistically through transcripts, stored Markdown state, and retrieval mechanisms.)

B. Channel-Access Boundary Risks

The channel-access boundary (F_1) constitutes the initial attack surface where external entities first obtain the ability to influence the OpenClaw orchestration core. In this architecture, channel ingress is not a benign presentation layer; it functions as the definitive gateway to a pipeline that dynamically populates session state, accumulates memory, and triggers tool execution. Consequently, the primary threat at this boundary transcends traditional account-level unauthorized access. The critical risk lies in exerting unauthorized, control-bearing influence over a stateful agent through ostensibly legitimate communication paths. This vulnerability aligns with broader agent-security empirical studies: during live red-teaming of autonomous agents possessing messaging access and file/shell permissions, researchers observed rampant identity spoofing, unauthorized compliance, and sensitive information disclosure immediately upon adversarial interaction reaching the agent interface [54].

Representative threats at this boundary include pairing-code interception, sender-admission bypass (e.g., allowFrom spoofing), authentication token theft, exposure of unauthenticated websocket surfaces, and direct prompt injection delivered via standard messaging channels. OpenClaw's internal threat model acknowledges pairing interception and token theft as severe access threats, while classifying crafted channel prompts as direct execution-path risks, given that malicious instructions easily bypass normal conversational filters. Recent systemic evaluations confirm this is not an anomaly isolated to OpenClaw. The Agent Security Bench (ASB) [17] reports exceptionally high attack success rates targeting the user-prompt handling mechanisms of tool-enabled agents. Furthermore, studies on instruction-data separation demonstrate that current Large Language Models (LLMs) fundamentally struggle to reliably distinguish benign conversational data from executable adversarial instructions [17], [55].

These vulnerabilities exist because OpenClaw centralizes routing, admission, and session continuity behind a singular gateway, implicitly concentrating trust at the communication ingress point. Once a malicious payload permeates this boundary, it rarely remains confined to the ingress layer. Instead, it is rapidly assimilated into the session context, polluting subsequent state evolution and steering downstream tool utilization [56]. This necessitates OpenClaw's strict operational guidance regarding authorized user pairing and rigorous Direct Message (DM) isolation in multi-user environments. Within our taxonomy, the channel-access boundary is therefore the critical beachhead: the primary infiltration point from which broader, cross-boundary attack propagation initiates.

C. Session-and-State Boundary Risks

The session-and-state boundary (F_2) governs the critical transformation of transient interactions into durable operational state. Within OpenClaw, this boundary encompasses session identity management, transcript history, memory-bearing context files (e.g., MEMORY.md), and dynamically updated configuration state. The fundamental security threat at this layer is persistence: the survival and amplification of attacker influence long after the initial interaction has terminated. Once adversarial or misleading content is ingested into session history or retrieved memory, it actively shapes future cognitive loops, effectively weaponizing the agent's own state against it.

This persistent threat model is heavily corroborated by recent agent-security literature. The ASB framework [17] identifies memory retrieval as a highly vulnerable operational stage, noting high attack success rates that easily bypass current defensive paradigms. Similarly, research on memory-augmented LLM agents demonstrates that even query-only, non-privileged interactions can successfully poison long-term memory, fundamentally altering subsequent agent responses [18], [19]. Representative threats manifesting at this boundary include cross-session data contamination, transcript leakage, targeted memory poisoning, context drift, and configuration tampering. These vulnerabilities emerge because modern architectures like OpenClaw do not treat historical data as passive logs; rather, past interaction traces are automatically re-injected as highly trusted context [57]. The *MemoryGraft* attack [58] perfectly illustrates this mechanism: by surreptitiously implanting strategically crafted, seemingly benign "successful experiences" into long-term memory, an attacker ensures that future retrievals surface unsafe behavioral patterns, inducing permanent behavioral drift across sessions. Relatedly, the *AgentSys* evaluation [59] observes that agents indiscriminately accumulating tool outputs and reasoning traces within working memory create a compounded vulnerability: malicious instructions persist indefinitely throughout the workflow, while irrelevant adversarial noise degrades overall decision quality.

The consequences of compromise at the session-and-state boundary are inherently cumulative. A poisoned session state can silently leak private context across users, deterministically bias future planning phases, or seamlessly hijack later tool-mediated actions without requiring continuous input from

the attacker. Consequently, this layer acts as the vital transmission belt between initial ingress and physical actuation: it is the exact juncture where short-lived attacker input metamorphoses into long-lived operational leverage. Securing this boundary demands far more than input sanitization; it requires rigorous cryptographic isolation of sessions, strict validation of memory write-back operations, and continuous semantic sanitization of retrieved state.

D. Tool-Execution Boundary Risks

The tool-execution boundary (F_3) marks the critical threshold where model-mediated cognitive intent is translated into operational authority. Within OpenClaw, this encompasses the transition from internal reasoning to concrete system actuations, including shell execution, browser automation, filesystem I/O, and remote procedure calls (RPCs) to connected nodes. Consequently, this is the exact juncture where upstream adversarial influence tangibly impacts the host environment. OpenClaw's internal threat model explicitly recognizes this layer as a distinct trust boundary, categorizing Tool Argument Injection and Execution Approval Bypass as primary execution-stage threats, with Unauthorized Command Execution representing a critical downstream impact. Broader agent-security benchmarks corroborate this assessment: the agent-tool interface functions as a primary security bottleneck, with empirical attack success rates exceeding 80% against tool-augmented agents [60].

Representative threats at this boundary include tool-argument manipulation, execution approval bypass, unauthorized command execution, and resource-amplification attacks. These vulnerabilities stem from the inherent fragility of translating natural-language context into structured parameters, URLs, and executable binaries. The *ToolHijacker* framework [61] demonstrates that prompt injection can deterministically manipulate tool selection, coercing the agent into invoking attacker-controlled tools—a vector against which both prevention and detection mechanisms remain highly ineffective. At the protocol layer, Model Context Protocol (MCP) integrations introduce severe architectural weaknesses, including missing capability attestation and implicit trust propagation [22]. Complementary analyses reveal that default MCP servers often execute with broad host access, necessitating strict declarative policy enforcement to mitigate the massive execution attack surface [23]. Thus, the danger transcends mere prompt "misinterpretation"; imperfect tool mediation actively converts adversarial text into system-level authority.

The consequences of compromise at this layer are immediate and systemic. OpenClaw's critical-path analysis delineates a direct escalation chain: from prompt injection, to approval bypass, concluding in unauthorized command execution. Furthermore, execution abuse need not be overtly destructive. Recent studies on tool-layer Economic Denial of Service (EDoS) reveal that malicious MCP servers can preserve task correctness while inflating per-query computational costs by up to 658×, forcing agents into parasitic, wasteful tool-calling loops that easily evade standard behavioral monitors [62], [63]. Ultimately, the tool-execution boundary is where informational attacks materialize as opera-

tional harm, underscoring the necessity of OpenClaw’s recommended mitigations: rigorous sandboxing, narrow cryptographic tool allowlists, and starkly partitioned access profiles.

E. External-Content Boundary Risks

The external-content boundary (F_4) governs the ingestion of unverified information asynchronously retrieved from outside the immediate communication channel. In OpenClaw, this encompasses parsed URLs, emails, documents, and web-hook payloads. The system’s threat model correctly treats this ingestion as a distinct trust boundary, mandating dedicated controls such as external-content wrapping and security notices. This architectural caution is heavily validated by seminal work on Indirect Prompt Injection (IPI): once untrusted external text is concatenated into an LLM’s prompt context, the semantic boundary between passive data and active instruction collapses, allowing seemingly benign content to hijack the agent’s control flow [20].

Representative threats manifesting at this boundary include indirect prompt injection, retrieval poisoning, tool-result injection, and external-content-mediated data exfiltration [64], [65]. OpenClaw’s threat matrix explicitly highlights content-wrapper escape, staged payload delivery, and data theft via the `web_fetch` tool as primary risks. The broader literature reinforces these vectors. *AgentDojo* [66] formalizes a comprehensive evaluation environment demonstrating how poisoned data returned by external tools successfully hijacks downstream agent behavior across email, banking, and workspace environments. Consequently, external content cannot be treated merely as passive knowledge; it must be handled as a potentially hostile instruction carrier [20].

These vulnerabilities emerge from three structural deficiencies in modern agent architectures. First, current LLMs fail to maintain robust data-control separation once retrieved text enters the semantic context window. Second, the retrieval process inherently transforms passive data into active influence by reintroducing it into the agent’s planning and execution loops. Third, successful exploits fundamentally disrupt task alignment. *Task Shield* [67] demonstrates that indirect attacks succeed precisely because the agent abandons the user’s original objective in favor of the injected payload. Furthermore, the *MELON* framework [68] proves that successful IPI causes future agent actions to overwhelmingly align with the malicious retrieved tasks rather than the genuine user request.

The consequences of compromise at this boundary are both immediate and highly propagative. Poisoned external content can systematically bias reasoning, distort planning algorithms, and silently redirect future tool invocations without requiring any direct malicious interaction from the attacker. Crucially, this injected payload is often written into session history or persistent memory, effectively bridging the F_4 boundary with the Session-and-State boundary (F_2). OpenClaw’s internal critical-chain analysis perfectly illustrates this cross-boundary propagation: “Indirect Injection Data Theft” proceeds sequentially from poisoned fetched content, to environment enumeration, culminating in active exfiltration via `web_fetch`. Thus, the external-content boundary serves not only as an independent vulnerability class but as a primary catalyst for down-

stream state corruption and physical execution harm.

F. Extension Supply-Chain Boundary Risks

The extension supply-chain boundary (F_5) governs how third-party capabilities infiltrate the agent ecosystem via publication, installation, and update mechanisms. In OpenClaw, the ClawHub registry [28] is not merely an ancillary feature; it constitutes a critical trust boundary because it injects externally authored instructions, execution scripts, and programmatic dependencies directly into the native runtime. Consequently, the central threat at this layer fundamentally diverges from passive content ingestion. Rather than merely manipulating context, compromised supply chains import novel execution paths, elevated privilege assumptions, and permanent malicious capabilities directly into the host system.

Representative threats at this boundary include malicious skill installation, compromised update pipelines, hidden instruction payloads embedded within skill metadata, credential harvesting, and capability escalation via bundled dependencies [69], [70]. Recent empirical evaluations confirm the severe reality of these vectors. The *SkillScan* framework [21] analyzed 42 447 skills across major marketplaces, revealing that 26.1% contained exploitable vulnerabilities ranging from prompt injection to data exfiltration. A complementary large-scale study [15] verified 98 380 skills and isolated 157 definitively malicious artifacts. These clustered heavily around two archetypes: “Data Thieves” designed to exfiltrate host credentials, and “Agent Hijackers” engineered to permanently subvert the agent’s decision-making logic.

These threats proliferate due to three structural vulnerabilities inherent to agentic supply chains. First, there is a severe semantic gap between description and execution: users trust natural-language descriptions or registry metadata, while the runtime blindly executes the underlying scripts [15]. Second, agent skills are hybrid threat artifacts—fusing textual specifications (SKILL.md) with executable code—rendering conventional static analysis and surface-level moderation highly inadequate. Third, the attack surface spans the entire artifact lifecycle [21]. Formal security analyses of AI skills demonstrate that dependency resolution, runtime installation, and subsequent background updates all present distinct exploitation windows, elevating post-install integrity to a primary security imperative [16].

Compromise at this boundary inherently bypasses the need for ephemeral prompt injection. Once a malicious skill is admitted, the attacker gains durable operational leverage: the installed artifact can autonomously harvest secrets, persist across isolated tasks, and fundamentally rewrite session state and memory. In this sense, the extension supply chain is a mechanism for converting a single external trust failure into permanent, systemic host compromise.

G. Cross-Boundary Threat Propagation

A fundamental axiom of the OpenClaw architecture is that security failures are rarely localized. Instead, adversarial influence actively propagates across the aforementioned boundaries—entering via channel ingress (F_1 / F_4), metastasizing within session state and memory (F_2), and ultimately materi-

TABLE III
CROSS-BOUNDARY THREAT PROPAGATION MATRIX WITHIN THE OPENCLAW ARCHITECTURE

Trust Boundary	Access & Entry	Content Ingestion	State Evolution	Tool Mediation	Execution & Impact
Channel-Access (F_1)	Pairing-code interception; allowFrom spoofing; token theft	Direct prompt injection via trusted messaging channels	Stolen-token persistence; poisoned context window	Sender-driven permission steering	Unauthorized actuations via delegated user authority
Session & State (F_2)	Unauthorized session discovery	Transcript ingestion into long-term files	Memory poisoning; context drift; config tampering	Biased context hijacks future tool selection	Transcript leakage; persistent cross-session compromise
Tool-Execution (F_3)	Malicious tool enumeration	Tool argument injection via prompt payload	Malicious actions queued in execution context	Exec approval bypass; MCP command injection	Unauthorized shell execution; data destruction; DoS
External-Content (F_4)	Exposed fetch surfaces	Indirect prompt injection from URLs, emails, & docs	Poisoned retrieved context flushes to memory	Retrieval-driven action steering	Data theft via web_fetch; host enumeration
Supply-Chain (F_5)	Malicious skill acting as Trojan entry point	Unvetted artifact installation from marketplace	Skill-based persistence; poisoned update delivery	Hidden tool exposure through rogue scripts	Credential harvesting; persistent backdoor execution

alizing as physical harm via tool-mediated execution (F_3). To systematize this dynamic, Table III presents a stage-wise threat propagation matrix, mapping our boundary-first taxonomy against the chronological evolution of an attack lifecycle. This propagation-centric perspective aligns with cutting-edge agent-security research [17], which demonstrates that realistic agent compromise rarely stems from a single exploit, but rather from the precise composition of multi-step attack chains.

Three representative cross-boundary propagation chains within OpenClaw are identified:

1) $F_1 \rightarrow F_2 \rightarrow F_3$ (**Direct Channel Compromise**): Malicious instructions admitted through ostensibly legitimate messages bypass the gateway (F_1), embed themselves into durable session context (F_2), and subsequently hijack downstream tool authorizations (F_3).

2) $F_4 \rightarrow F_2 \rightarrow F_3$ (**External Retrieval Hijacking**): Poisoned webpages or documents are asynchronously fetched (F_4), corrupting the agent's working memory (F_2). As demonstrated by the *Task Shield* framework [67], this silently realigns the agent's action trajectory away from the user's objective and triggers unauthorized tool execution or exfiltration (F_3).

3) $F_5 \rightarrow F_2 \rightarrow F_3$ (**Supply-Chain Subversion**): A malicious skill embeds harmful capability directly into the environment (F_5). This artifact does not merely inject transient instructions; it permanently alters how the agent processes future state (F_2) and fundamentally expands the blast radius of available tools (F_3) across all subsequent sessions [18].

Recent OpenClaw-specific cases provide practical evidence for this propagation model. One representative case is the indirect prompt-injection attack demonstrated against OpenClaw through a malicious webpage [71]. In this case, attacker-controlled web content was fetched into the agent context, induced the agent to execute a remote installation script through the exec tool, and then modified HEARTBEAT.md so that malicious instructions persisted across later sessions. This case closely matches the $F_4 \rightarrow F_2 \rightarrow F_3$ chain in our taxonomy: untrusted external content first entered the context through web retrieval, then became durable operational state through a workspace memory file, and finally obtained sys-

tem-level effect through tool-mediated command execution. A second case concerns malicious OpenClaw skills used to distribute Atomic macOS Stealer (AMOS) [72]. In this campaign, malicious instructions hidden in SKILL.md files manipulated agentic workflows into installing a fake command-line tool, which then downloaded and executed a payload that collected local files, credentials, keychain data, and user documents before uploading them to a command-and-control server. This case directly reflects the $F_5 \rightarrow F_2 \rightarrow F_3$ chain: an unvetted third-party skill is admitted from the extension ecosystem, its instructions become part of the agent's operational context, and the resulting execution path obtains local-system authority and exfiltrates sensitive data. Together, these cases show that cross-boundary propagation is not only a conceptual risk, but a practically demonstrated failure pattern in OpenClaw-like deployments.

This fluid propagation is driven by three architectural realities. First, because OpenClaw utilizes a shared semantic control channel, attacker intent easily translates from passive text to active state, and finally to executable action. Second, persistent memory architectures ensure that transient adversarial influence survives long beyond the original interaction [18]. Third, tool-enabled autonomy bridges the actuation gap, guaranteeing that upstream cognitive misalignment is inevitably translated into real-world operational harm rather than remaining a benign interpretive error [73].

The paramount implication for systems security is that boundary-specific mitigations are strictly necessary but inherently insufficient. Because attacks chain across domains, effective defense architectures must employ holistic kill-chain analysis—evaluating how ingress, state persistence, retrieval, and execution interact dynamically over time.

VI. MITIGATION STRATEGIES: A BOUNDARY-AWARE DEFENSE ARCHITECTURE

Risk mitigation within OpenClaw must pivot away from the futile pursuit of completely eliminating prompt injection. Instead, the strategic imperative is to re-establish robust trust boundaries across the agent lifecycle, ensuring that mixed-trust inputs, persistent state, tool authority, and external extensions are no longer fused into a single uncontrolled opera-

tional loop. Recent scholarship asserts that agent security is inherently contextual, heavily reliant on source authorization, task alignment, and strict data isolation rather than static keyword filtering alone [24]. Concurrently, secure-design research for LLM-integrated systems emphasizes defense-in-depth, least privilege, complete mediation, and extension governance as baseline engineering principles. For OpenClaw, this necessitates a paradigm shift: from merely detecting malicious text toward deterministically controlling how channel inputs mutate into memory, how memory dictates planning, and how plans acquire system-level authority.

Securing the ingress (F_1) and state (F_2) layers requires preventing transient adversarial influence from metastasizing into durable operational context. Source authorization must evolve beyond simple access control to include cryptographic attestation of safeguard execution. Frameworks like *Proof-of-Guardrail* [74] demonstrate that guardrail execution can be verified via Trusted Execution Environments (TEEs), providing mathematical assurance that inputs were sanitized prior to ingestion. Within the runtime, persistent memory must be heavily guarded rather than acting as an indiscriminate sink for transcript fragments. *AgentSys* [59] pioneers a robust approach by isolating tool-facing worker contexts from the primary agent, permitting only schema-validated returns to cross memory boundaries. Furthermore, *DRIFT* [75] and *Contextualized Privacy Defense* [76] extend this by dynamically isolating injection-bearing memory streams and enforcing step-specific privacy guidance. For OpenClaw, the architectural implication is profound: transcripts and retrieved memory must never flow directly into the main reasoning context without passing through strict sanitization, scoped persistence mechanisms, and explicit write-back policies.

The paramount choke point for systemic security, however, remains the Tool-Execution boundary (F_3). The defensive objective is not merely to reject dangerous prompts, but to prevent untrusted context from acquiring privileged actuation authority over the host environment. *Task Shield* [67] provides a foundational principle: intermediate instructions and tool calls must be continuously evaluated for alignment with the user's original objective, rather than judged solely by their local surface form. *IPIGuard* [77] complements this by decoupling action planning from external data interaction via a Tool Dependency Graph, effectively neutralizing unintended invocations. At the enforcement layer, *Progent* [78], *MCP-Secure* [23], and *AC4A* [79] converge on a definitive design pattern: tool calls must be intercepted pre-execution, evaluated against fine-grained declarative policies, and permitted exclusively under strict capability constraints. OpenClaw deployments must therefore operationalize bounded tool permissions, argument-level validation, and mandatory Human-in-the-Loop (HITL) approvals for high-risk operations [80], establishing layered execution control without requiring modifications to the underlying LLM weights.

Mitigating risks at the External-Content (F_4) and Extension Supply-Chain (F_5) boundaries requires strict containment paradigms. Externally fetched webpages and documents must be aggressively sandboxed from the semantic control plane. They should be treated as untrusted, control-bearing inputs

through content wrapping, provenance tagging, and isolated parsing stages before they are permitted to influence downstream tool calls [20], [66]–[68]. Similarly, the governance of ClawHub skills must shift from permissive installation to strict capability bounding. Skills must be treated as untrusted binaries, necessitating pre-install vetting, permission-minimized packaging, integrity-protected updates, and rigorous runtime containment via sandboxing [16], [70], [74], [80]. Because skills directly expand OpenClaw's reachable tool and data surfaces, admitting them without zero-trust enforcement guarantees persistent runtime compromise.

Ultimately, mitigation within OpenClaw must prioritize resilience, observability, and deterministic recovery over the illusion of perfect prevention [81], [82]. Security auditing, transcript review, and deliberate restrictions on agent interfaces are non-negotiable operational requirements [13]. From an application testing engineering perspective, recent work on LLM-integrated applications also suggests that practical defenses should be evaluated not only by security effectiveness, but also by their latency and usability impact in real-world deployments [83]. Because cross-boundary attacks rarely fail cleanly at a single node, they are most effectively detected through anomalous state changes, widened permissions, or highly irregular tool-call trajectories. Consequently, the most credible defense-in-depth strategy for OpenClaw is a coordinated, boundary-aware architecture that seamlessly integrates cryptographic access control, state isolation, pre-execution mediation, supply-chain governance, and post-hoc auditability across the entire agent lifecycle.

VII. OPEN CHALLENGES AND FUTURE DIRECTIONS

While the boundary-aware defense architecture constructed in Section VI substantially hardens OpenClaw, the paradigm shift toward persistent, gateway-mediated, and highly extensible agents introduces foundational security challenges that remain unresolved. Because OpenClaw-like systems transcend stateless prompt processing—fusing untrusted ingress, durable memory, physical actuation, and third-party capabilities into a continuous runtime—future research must pivot from isolated prompt-level heuristics toward systemic, cryptographically verifiable guarantees.

Foremost among these is the **absence of a unified security semantics for agentic systems**. Action legitimacy is inherently contextual: a specific tool invocation may be benign or catastrophic depending on the initiating entity, the authorized objective, and the current environmental state. Consequently, security cannot be reduced to static keyword filters or generic jailbreak detectors [24]. Future research must formalize contextual security properties—such as source authorization, task alignment, and strict data isolation—and compile them into deterministic runtime constraints. Bridging this semantic gap is critical to resolving the persistent utility-security tradeoff, ensuring defenses are grounded in the rigorous semantics of authorized behavior rather than blanket operational degradation.

Applying these semantics to the system perimeter, a critical localized challenge lies in **authenticated channel ingress and provenance enforcement** (F_1). In OpenClaw-like archi-

tures, the channel boundary is not a passive user-interface layer, but the definitive trust gate through which external entities acquire the ability to manipulate memory and orchestrate execution. Given that sender spoofing, pairing-code interception, and direct prompt injection establish a practical beachhead for cross-boundary compromise, future research must move beyond coarse allowlists. Critical vectors include cryptographically verifiable channel authentication, provenance-preserving message transport, and deterministic session-binding integrity, ensuring that later defenses do not operate on already-compromised context [54].

Compounding this semantic gap is the critical need for **long-horizon, stateful evaluation frameworks**. While existing benchmarks have catalyzed agent security research, they predominantly evaluate short-horizon, stateless, or easily resettable environments [17], [84]. OpenClaw's architecture, characterized by persistent memory and continuous sessions, dictates that risks accumulate and compound over time. Consequently, one-shot exploit evaluations are fundamentally inadequate. Future benchmarking paradigms must simulate longitudinal attack chains, mixed-trust interaction histories, and delayed-trigger payloads under black-box conditions. These frameworks must quantify not just immediate bypass success, but longitudinal behavioral drift, cross-session compromise, and the difficulty of post-breach recovery.

At the architectural level, **persistent memory governance** (F_2) remains a deeply unresolved vector. In OpenClaw, memory is not a passive retrieval archive; it is a highly active operational substrate that preserves adversarial influence and deterministically biases future planning [18], [19], [58], [59]. Current mitigations rely heavily on retrieval-time filtering, which is systematically insufficient for long-lived assistants [85]. Future research must pioneer secure memory write-back protocols, causality-aware retrieval algorithms, cryptographic state provenance, and mechanisms for verifiable "selective forgetting." Memory architectures must evolve to actively isolate trustworthy state from adversarial residue, making rigorous state evolution a primary research frontier.

Bridging this memory state to physical actuation is the unresolved challenge of **trustworthy mediation of retrieved external content** (F_4). In OpenClaw, asynchronously fetched webpages, emails, and tool-returned text do not remain passive data; they often mutate into control-bearing inputs that reshape planning and subsequent execution. Future work must rigorously investigate mechanisms for enforcing data-instruction separation post-retrieval, isolating unverified external observations from high-authority planning states, and strictly constraining how retrieved payloads influence downstream actions. Because poisoned external content frequently cascades into persistent memory (F_2) and tool invocation (F_3), neutralizing this boundary is essential for halting cross-boundary attack propagation [66], [67].

Equally critical is the transition toward **verifiable, mathematically constrained tool execution** (F_3). OpenClaw derives its utility from translating cognitive outputs into host-level actuation. Existing research highlights the necessity of task-aligned mediation and privilege-aware policies [23], [67], [78], [79]. However, contemporary approaches offer only

probabilistic guarantees, particularly when tool calls are dynamically chained or routed via complex interoperability protocols [22], [80]. Future investigations must explore formal specification languages for tool-use policies, rigorous argument-level type verification, and secure protocol composition. The discipline must advance from best-effort anomaly detection to deterministic, pre-execution capability constraints.

The proliferation of third-party capabilities introduces the challenge of **securing agentic supply chains and extension ecosystems** (F_5). OpenClaw's extensibility fundamentally expands the Trusted Computing Base (TCB). Empirical analyses reveal that staggering percentages of real-world skills contain exploitable vulnerabilities, credential-harvesting routines, or instruction-hijacking payloads [15], [16], [21], [70]. Scalable defenses for these hybrid textual-executable artifacts remain nascent. Future research must establish zero-trust extension architectures, focusing on attestable code provenance, permission-minimized packaging paradigms, cryptographically signed updates, and dynamic sandboxing. Trust cannot be inferred from registry metadata; it must be continuously verified at runtime.

Finally, the paradigm of continuous autonomous execution necessitates breakthroughs in **cryptographic auditability and scalable human oversight**. Current reliance on rudimentary logs, heuristic alerts, and fatigue-inducing Human-in-the-Loop (HITL) prompts is unsustainable for reconstructing complex, multi-step agent failures. Emerging work on Trusted Execution Environment (TEE) guardrail attestation points toward a necessary evolution: security controls must produce verifiable evidence, not mere logging assertions [74], [80], [86]. Future systems must generate immutable, audit-ready telemetry spanning cognitive planning traces, state transitions, and tool actuations, enabling rigorous post-hoc forensic inspection without overwhelming human operators.

In synthesis, these open challenges confirm that the security of continuous, self-hosted agent platforms like OpenClaw cannot be solved via isolated prompt engineering. The research community must execute a structural pivot toward context-aware security semantics, longitudinal threat evaluation, verifiable state governance, deterministic execution constraints, zero-trust extensibility, and cryptographically sound auditability. Advancing these frontiers will be paramount in transitioning autonomous LLM agents from experimentally hardened prototypes to highly resilient, security-assurable operational platforms.

VIII. CONCLUSION

This paper presented a security-oriented study of OpenClaw as a representative autonomous self-hosted AI agent platform. Rather than treating threats as isolated vulnerabilities, OpenClaw was analyzed through a trust-boundary-first lens that captures how adversarial influence propagates over time across channel access, session/state management, tool execution, external content ingestion, and extension supply chains. The analysis showed that major risks, including indirect prompt injection, memory poisoning, unauthorized tool use, data exfiltration, and malicious skill abuse, are best

understood as cross-boundary escalation processes in long-lived agent lifecycles rather than as one-shot prompt failures. Based on this perspective, key defense-in-depth priorities for practical deployments were identified, including stricter boundary separation, capability-scoped and verifiable tool mediation, memory integrity governance, extension trust controls, and audit-ready operational oversight. More broadly, OpenClaw illustrates a broader transition in agent security research from model-centric robustness to systems-level security engineering for persistent autonomous runtimes. It is hoped that this work will provide both an actionable analysis framework for practitioners and a foundation for future research on building secure, observable, and recoverable autonomous AI agent systems.

REFERENCES

- [1] V. Raval, M. Zeid, and P. Enjeti, "Circuit-AI: A self-hosted AI-agent language model framework for control loop implementation and simulation," in *Proc. IEEE Int. Communications Energy Conf.*, Houston, USA, 2025, pp. 91–96.
- [2] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, Feb. 2025.
- [3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024.
- [4] W. Zhou, X. Zhu, Q.-L. Han, L. Li, X. Chen, S. Wen, and Y. Xiang, "The security of using large language models: A survey with emphasis on ChatGPT," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 1, pp. 1–26, Jan. 2025.
- [5] Z. Deng, R. Sun, M. Xue, W. Ma, S. Wen, S. Nepal, and Y. Xiang, "Hardening LLM fine-tuning: From differentially private data selection to trustworthy model quantization," *IEEE Trans. Inform. Foren. Sec.*, vol. 20, pp. 7211–7226, Jun. 2025.
- [6] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang, "AI agents under threat: A survey of key security challenges and future pathways," *ACM Comput. Surv.*, vol. 57, no. 7, Art. no. 182, Feb. 2025.
- [7] Z. Li, W. Wu, Y. Guo, J. Sun, and Q.-L. Han, "Embodied multi-agent systems: A review," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 6, pp. 1095–1116, Jun. 2025.
- [8] P. Steinberger, "OpenClaw: Your own personal AI assistant," 2026. [Online]. Available: <https://github.com/openclaw/openclaw>.
- [9] LangChain AI, "LangGraph: Build resilient language agents as graphs," 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>.
- [10] CrewAI Inc., "CrewAI," 2025. [Online]. Available: <https://github.com/crewAIInc/crewAI>.
- [11] LlamaIndex, "LlamaIndex," 2025. [Online]. Available: <https://www.llamaindex.ai/>.
- [12] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, "Taming various privilege escalation in LLM-based agent systems: A mandatory access control framework," arXiv preprint arXiv: 2601.11893, 2026.
- [13] OWASP Cheat Sheets Series Team, "AI agent security cheat sheet," 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/AI_AgentSecurityCheatSheet.html. Accessed on: Mar. 22, 2026.
- [14] Z. Deng, W. Ma, Q.-L. Han, W. Zhou, X. Zhu, S. Wen, and Y. Xiang, "Exploring DeepSeek: A survey on advances, applications, challenges and future directions," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 5, pp. 872–893, May 2025.
- [15] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, and L. Y. Zhang, "Malicious agent skills in the wild: A large-scale security empirical study," arXiv preprint arXiv: 2602.06547, 2026.
- [16] Y. Jiang, D. Li, H. Deng, B. Ma, X. Wang, Q. Wang, and G. Yu, "SoK: Agentic skills-beyond tool use in LLM agents," arXiv preprint arXiv: 2602.20867, 2026.
- [17] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *Proc. 13th Int. Conf. Learning Representations*, Singapore, Singapore, 2025.
- [18] B. D. Sunil, I. Sinha, P. Maheshwari, S. Todmal, S. Mallik, and S. Mishra, "Memory poisoning attack and defense on memory based LLM-agents," arXiv preprint arXiv: 2601.05504, 2026.
- [19] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. Xiang, "Memory injection attacks on LLM agents via query-only interaction," in *Proc. 39th Annu. Conf. Neural Information Processing Systems*, 2025.
- [20] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, Copenhagen, Denmark, 2023, pp. 79–90.
- [21] Y. Liu, W. Wang, R. Feng, Y. Zhang, G. Xu, G. Deng, Y. Li, and L. Zhang, "Agent skills in the wild: An empirical study of security vulnerabilities at scale," arXiv preprint arXiv: 2601.10338, 2026.
- [22] A. Doshi, Y. Hong, C. Xu, E. Kang, A. Kapravelos, and C. Kästner, "Towards verifiably safe tool use for LLM agents," in *Proc. Int. Conf. Software Engineering, New Ideas and Emerging Results Track*, 2026.
- [23] G. Singh and V. K. Madiseti, "MCP-secure: A runtime access control layer for privilege-aware LLM agent tooling," *IEEE Open J. Comput. Soc.*, vol. 7, pp. 504–513, Feb. 2026.
- [24] Z. Li, W. Li, and X. Li, "Defensible design for OpenClaw: Securing autonomous tool-invoking agents," arXiv preprint arXiv: 2603.13151, 2026.
- [25] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proc. 34th USENIX Conf. Security Symp.*, Seattle, USA, 2025, Art. no. 123.
- [26] W. Ma, Y. Guo, Q.-L. Han, W. Zhou, X. Zhu, J. Xiong, S. Wen, and Y. Xiang, "Understanding agentic AI: Algorithms and infrastructure," *IEEE/CAA J. Autom. Sinica*, vol. 13, no. 4, pp. 776–795, 2026.
- [27] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inform. Retr.*, vol. 3, no. 4, pp. 333–389, Apr. 2009.
- [28] P. Steinberger, "ClawHub," [Online]. Available: <https://clawhub.ai/>.
- [29] badlogic, "Pi monorepo," 2026. [Online]. Available: <https://github.com/badlogic/pi-mono>. Accessed on: Mar. 22, 2026.
- [30] IBM BeeAI, "Agent communication protocol," 2026. [Online]. Available: <https://agentcommunicationprotocol.dev>. Accessed on: Mar. 22, 2026.
- [31] Autobot, "Ultra-efficient personal AI assistant powered by crystal," [Online]. Available: <https://github.com/crystal-autobot/autobot>. Accessed on: Mar. 22, 2026.
- [32] Pickle-Bot, "Your own AI assistant," 2026. [Online]. Available: <https://github.com/czl9707/pickle-bot>. Accessed on: Mar. 22, 2026.
- [33] Moxxy, "Your everyday AI assistant agents," 2026. [Online]. Available: <https://github.com/moxxy-ai/moxxy>. Accessed on: Mar. 22, 2026.
- [34] TinyAGI, "TinyAGI is the agent teams orchestrator for one person company (fka TinyClaw)," 2026. [Online]. Available: <https://github.com/TinyAGI/tinyagi>. Accessed on: Mar. 22, 2026.
- [35] HermitClaw, "A tiny AI creature that lives in a folder on your computer," [Online]. Available: <https://github.com/brendanhogan/hermitclaw>. Accessed on: Mar. 22, 2026.
- [36] Nanobot, "Ultra-lightweight personal AI assistant," [Online]. Available: <https://github.com/HKUDS/nanobot>. Accessed on: Mar. 22, 2026.
- [37] OpenFang, "The agent operating system," 2026. [Online]. Available: <https://github.com/RightNow-AI/openfang>. Accessed on: Mar. 22, 2026.
- [38] AstrBot, "Agentic IM chatbot infrastructure," [Online]. Available: <https://github.com/AstrBotDevs/AstrBot>. Accessed on: Mar. 22, 2026.
- [39] ZeroClaw, "Personal AI assistant," [Online]. Available: <https://github.com/zeroclaw-labs/zeroclaw>. Accessed on: Mar. 22, 2026.
- [40] SupaClaw, "Built entirely on Supabase built-in features," [Online]. Available: <https://github.com/vincenzodomina/supaclaw>. Accessed on: Mar. 22, 2026.
- [41] LettaBot, "Your personal AI assistant that remembers everything," [Online]. Available: <https://github.com/letta-ai/lettabot>. Accessed on: Mar. 22, 2026.
- [42] Flowly, "Your personal AI assistant, one click away," [Online]. Available: <https://www.useflowlyapp.com/en>. Accessed on: Mar. 22, 2026.
- [43] DroidClaw, "An AI agent that controls your android phone," 2026. [Online]. Available: <https://github.com/unitedbyai/droidclaw>. Accessed

- on: Mar. 22, 2026.
- [44] BabyClaw, “Claude code on a VPS, controlled from telegram,” [Online]. Available: <https://github.com/yogesharc/babyclaw>. Accessed on: Mar. 22, 2026.
 - [45] Zclaw, “Your personal AI assistant,” 2026. [Online]. Available: <https://github.com/tnm/zclaw>. Accessed on: Mar. 22, 2026.
 - [46] MimiClaw, “MimiClaw: Pocket AI assistant on a \$5 chip,” 2026. [Online]. Available: <https://github.com/memovai/mimiclaw>. Accessed on: Mar. 22, 2026.
 - [47] Picobot, “The AI agent that runs anywhere — even on a \$5 VPS,” 2026. [Online]. Available: <https://github.com/louisho5/picobot>. Accessed on: Mar. 22, 2026.
 - [48] PicoClaw, “PicoClaw: Ultra-efficient AI assistant in go,” 2026. [Online]. Available: <https://github.com/sipeed/picoclaw>. Accessed on: Mar. 22, 2026.
 - [49] AngelClaw, “The multi-tenant agent operating system,” 2026. [Online]. Available: <https://github.com/Abdur-rahmaanJ/angel-claw>. Accessed on: Mar. 22, 2026.
 - [50] SafeClaw, “The zero-cost alternative to OpenClaw,” 2026. [Online]. Available: <https://github.com/princezuda/safeclaw>. Accessed on: Mar. 22, 2026.
 - [51] Shrew, “Shrew: Ultra-lightweight agent,” 2026. [Online]. Available: <https://github.com/Masmedeam/shrew>. Accessed on: Mar. 22, 2026.
 - [52] IronClaw, “Your secure personal AI assistant, always on your side,” 2026. [Online]. Available: <https://github.com/neirai/ironclaw>. Accessed on: Mar. 22, 2026.
 - [53] Moltis, “A rust-native claw you can trust,” 2026. [Online]. Available: <https://github.com/moltis-org/moltis>. Accessed on: Mar. 22, 2026.
 - [54] N. Shapira, C. Wendler, A. Yen, G. Sarti, K. Pal, O. Floody, A. Belfki, A. Loftus, A. R. Jannali, N. Prakash, *et al.*, “Agents of chaos,” arXiv preprint arXiv: 2602.20021, 2026.
 - [55] E. Zverev, S. Abdelnabi, S. Tabesh, M. Fritz, and C. H. Lampert, “Can LLMs separate instructions from data? And what do we even mean by that?” in *Proc. 13th Int. Conf. Learning Representations*, Singapore, Singapore, 2025.
 - [56] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Proc. Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, 2024, pp. 10471–10506.
 - [57] X. Yang, Y. He, S. Ji, B. Hooi, and J. S. Dong, “Zombie agents: Persistent control of self-evolving LLM agents via self-reinforcing injections,” arXiv preprint arXiv: 2602.15654, 2026.
 - [58] S. S. Srivastava and H. He, “MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval,” arXiv preprint arXiv: 2512.16962, 2025.
 - [59] R. Wen, H. Li, C. Xiao, and N. Zhang, “AgentSys: Secure and dynamic LLM agents through explicit hierarchical memory management,” arXiv preprint arXiv: 2602.07398, 2026.
 - [60] Y. Liu, Y. Cui, and H. Zhang, “RRTL: Red teaming reasoning large language models in tool learning,” *IEEE Internet Things J.*, vol. 13, no. 6, pp. 11868–11880, Mar. 2026.
 - [61] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, “Prompt injection attack to tool selection in LLM agents,” in *Proc. 33rd Annu. Network and Distributed System Security Symp.*, San Diego, USA, 2026.
 - [62] B. Dong, H. Feng, and Q. Wang, “Clawdrain: Exploiting tool-calling chains for stealthy token exhaustion in OpenClaw agents,” arXiv preprint arXiv: 2603.00902, 2026.
 - [63] K. Zhou, Y. Zheng, Y. He, M. Xue, X. Gong, Y. Wang, and K.-Y. Lam, “Beyond max tokens: Stealthy resource amplification via tool calling chains in LLM agents,” arXiv preprint arXiv: 2601.10955, 2026.
 - [64] H. Chang, E. Bao, X. Luo, and T. Yu, “Overcoming the retrieval barrier: Indirect prompt injection in the wild for LLM systems,” in *Proc. ACM on Web Conf.*, 2026.
 - [65] X. Wang, J. Bloch, Z. Shao, Y. Hu, S. Zhou, and N. Z. Gong, “Webinject: Prompt injection attack to web agents,” in *Proc. Conf. Empirical Methods in Natural Language Processing*, Suzhou, China, 2025, pp. 2010–2030.
 - [66] E. DeBenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” *Proc. 38th Int. Conf. Neural Information Processing Systems*, Vancouver, Canada, 2024, Art. no. 2636.
 - [67] F. Jia, T. Wu, X. Qin, and A. Squicciari, “The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents,” in *Proc. 63rd Annu. Meeting Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria, 2025, pp. 29680–29697.
 - [68] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, “Melon: Provable defense against indirect prompt injection attacks in AI agents,” in *Proc. 42nd Int. Conf. Machine Learning*, Vancouver, Canada, 2025, pp. 80310–80329.
 - [69] F. Liu, Z. Chen, T. Lan, H. Tan, Z. Xu, X. Li, G. Chen, Y. Meng, and H. Zhu, “Trojan’s whisper: Stealthy manipulation of OpenClaw through injected bootstrapped guidance,” arXiv preprint arXiv: 2603.19974, 2026.
 - [70] Z. Guo, Z. Chen, X. Nie, J. Lin, Y. Zhou, and W. Zhang, “SkillProbe: Security auditing for emerging agent skill marketplaces via multi-agent collaboration,” arXiv preprint arXiv: 2603.21019, 2026.
 - [71] C. McCauley, K. Schulz, R. Tracey, and J. Martin, “Exploring the security risks of AI assistants like OpenClaw,” [Online]. Available: <https://www.hiddenlayer.com/research/exploring-the-security-risks-of-ai-assistants-like-openclaw>. Accessed on: Apr. 26, 2026.
 - [72] A. Oliveira, B. Tancio, D. Fiser, P. Lin, and R. Reyes, “Malicious OpenClaw skills used to distribute atomic MACOs stealer,” [Online]. Available: https://www.trendmicro.com/en_us/research/26/b/openclaw-skills-used-to-distribute-atomic-macos-stealer.html. Accessed on: Apr. 26, 2026.
 - [73] X. Deng, Y. Zhang, J. Wu, J. Bai, S. Yi, Z. Zou, Y. Xiao, R. Qiu, J. Ma, J. Chen, *et al.*, “Taming OpenClaw: Security analysis and mitigation of autonomous LLM agent threats,” arXiv preprint arXiv: 2603.11619, 2026.
 - [74] X. Jin, M. Duan, Q. Lin, A. Chan, Z. Chen, J. Du, and X. Ren, “Proof-of-guardrail in AI agents and what (not) to trust from it,” arXiv preprint arXiv: 2603.05786, 2026.
 - [75] H. Li, X. Liu, C. H. Chun, D. Li, N. Zhang, and C. Xiao, “DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents,” in *Proc. 39th Annu. Conf. Neural Information Processing Systems*, 2025.
 - [76] Y. Wen, Y. Zhang, J. Lian, X. Yi, X. Xie, and D. Yang, “Contextualized privacy defense for LLM agents,” arXiv preprint arXiv: 2603.02983, 2026.
 - [77] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, “IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents,” in *Proc. Conf. Empirical Methods in Natural Language Processing*, Suzhou, China, 2025, pp. 1023–1039.
 - [78] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, “Progent: Securing AI agents with privilege control,” arXiv preprint arXiv: 2504.11703, 2025.
 - [79] R. K. Sharma and D. Grossman, “AC4A: Access control for agents,” arXiv preprint arXiv: 2603.20933, 2026.
 - [80] Z. Shan, J. Xin, Y. Zhang, and M. Xu, “Don’t let the claw grip your hand: A security analysis and defense framework for OpenClaw,” arXiv preprint arXiv: 2603.10387, 2026.
 - [81] B. Rombaut, S. Masoumzadeh, K. Vasilevski, D. Lin, and A. E. Hassan, “Watson: A cognitive observability framework for the reasoning of LLM-powered agents,” in *Proc. 40th IEEE/ACM Int. Conf. Automated Software Engineering*, Seoul, Republic of Korea, 2025, pp. 739–751.
 - [82] A. Kaplunovich, “Advancing LLM agents for code generation: Observability, orchestration, reliable performance,” in *Proc. Int. Conf. Intelligent Computing, Communication, Networking and Services*, Varna, Bulgaria, 2025, pp. 108–116.
 - [83] Q. Lan, A. Kaul, and S. Jones, “Prompt injection detection in LLM integrated applications,” *Int. J. Network Dynam. Intell.*, vol. 4, no. 2, Art. no. 100013, 2025.
 - [84] Y. Wang, F. Xu, Z. Lin, G. He, Y. Huang, H. Gao, Z. Niu, S. Lian, and Z. Liu, “From assistant to double agent: Formalizing and benchmarking attacks on OpenClaw for personalized local AI agent,” arXiv preprint arXiv: 2602.08412, 2026.
 - [85] Y. Zhao, B. Yuan, J. Huang, H. Yuan, Z. Yu, L. Hu, H. Xu, A. Shankarampeta, Z. Huang, W. Ni, *et al.*, “AMA-Bench: Evaluating long-horizon memory for agentic applications,” in *Proc. ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems*, 2026.
 - [86] A. AlSayyad, K. Y. Huang, and R. Pal, “AgentTrace: A structured logging framework for agent system observability,” in *Proc. LLM-based Multi-Agent Systems: Towards Responsible, Reliable, and Scalable Agentic Systems*, 2026.



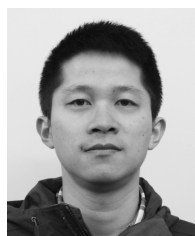
Wanlun Ma (Member, IEEE) received the Bachelor's and Master's degrees in information and communication engineering from the University of Electronic Science and Technology of China (UESTC) in 2017 and 2020, respectively; the Ph.D. degree in computer science from Swinburne University of Technology, Australia, in 2024. He is currently a Research Fellow with Swinburne University of Technology. His research interests focus on trustworthy and responsible AI, adversarial machine learning, and network security and privacy.



Qing-Long Han (Fellow, IEEE) received the B.Sc. degree in mathematics from Shandong Normal University in 1983, and the M.Sc. and Ph.D. degrees in control engineering from East China University of Science and Technology, in 1992 and 1997, respectively. Professor Han is Pro Vice-Chancellor (Research Quality) and a Distinguished Professor at Swinburne University of Technology, Melbourne, Australia. He held various academic and management positions at Griffith University and Central Queensland University, Australia. His research interests include networked control systems, multi-agent systems, time-delay systems, smart grids, unmanned surface vehicles, and neural networks.

Professor Han was awarded the 2024 IEEE Dr.-Ing. Eugene Mittelmann Achievement Award (the Highest Award in industrial electronics), the 2024 Chinese Association of Automation (CAA) Science and Technology Achievement Award (the Highest Achievement Award of CAA in Automation, Information and Intelligent Science), the 2021 Norbert Wiener Award (the Highest Award in systems science and engineering, and cybernetics) and the 2021 M. A. Sargent Medal (the Highest Award of the Electrical College Board of Engineers Australia). He was the recipient of the IEEE Systems, Man, and Cybernetics Society Andrew P. Sage Best Transactions Paper Award in 2019, 2020, and 2022, respectively, the IEEE/CAA Journal of Automatica Sinica Norbert Wiener Review Award in 2020, and the IEEE Transactions on Industrial Informatics Outstanding Paper Award in 2020.

Professor Han is a Member of the Academia Europaea (The Academy of Europe). He is a Fellow of the International Federation of Automatic Control (FIFAC), a Fellow of the Asian Control Association (FACA), an Honorary Fellow of the Institution of Engineers Australia (HonFIEAust), and a Fellow of the Chinese Association of Automation (FCAA). He is a Highly Cited Researcher in both Engineering and Computer Science (Clarivate). He has served as an AdCom Member of IEEE Industrial Electronics Society (IES), a Member of IEEE IES Fellows Committee, a Member of IEEE IES Publications Committee, Chair of IEEE IES Technical Committee on Network-Based Control Systems and Applications, and the Co-Editor-in-Chief of *IEEE Transactions on Industrial Informatics*. He is currently the President-Elect, an Executive Board Member, and a Steering Committee Member of Asian Control Association (ACA), and the Vice-President of the Chinese Association of Automation (CAA). He is currently the Editor-in-Chief of *IEEE/CAA Journal of Automatica Sinica* and the Co-Editor of *Australian Journal of Electrical and Electronic Engineering*.

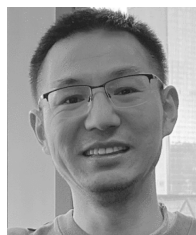


Xiaogang Zhu (Member, IEEE) received his Ph.D. degree in computer science and engineering at Swinburne University of Technology, Australia, in 2021. Currently, he is a Lecturer at the Adelaide University, Australia, and focuses on searching vulnerabilities in programs. He is interested in detecting techniques such as fuzzing, machine learning and symbolic execution. He has published papers on top journals, such as *IEEE Transactions on Dependable and Secure Computing* (TDSC), and conferences such as

IEEE Symposium on Security and Privacy (S&P), ACM Conference on Computer and Communications Security (CCS), USENIX Security Symposium (USENIX Security), and International Conference on Software Engineering (ICSE). He also served as a reviewer for many top journals such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Internet of Things Journal*, *ACM Computing Surveys*.



Wei Zhou (Member, IEEE) received the BEng and MEng degrees from Central South University in 2005 and 2008, respectively, all in computer science, and the Ph.D. degree from School of Engineering and IT, University of New South Wales, Canberra, Australia, in 2016. Her research interests include computer networks, network security, and fingerprint biometric.



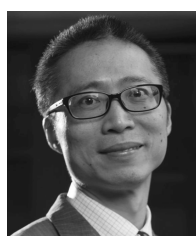
Junwu Xiong received a Master's degree from Beihang University. He currently works in JD Technology as Senior Director responsible for cloud platform, technology research and engineering management. His research interests include reinforcement learning, LLM and VLA RL post-training, agentic RL, world models, video and Omni generative foundation model reasoning, effective and efficient universal intelligent infrastructure. He has published multiple papers at the top international conferences including International Conference on Learning Representations (ICLR), International Conference on Machine Learning (ICML), Conference on Neural Information Processing Systems (NeurIPS), and AAAI Conference on Artificial Intelligence (AAAI).



Zhihang Ren received his Ph.D. degree from University of California, Berkeley, USA, in 2024. He currently works in Google LLC as a software engineer for core machine learning (ML) models and scalable ML training infrastructure. He is also a Research Affiliate at University of California, Berkeley, USA. His research interests include image and video processing, machine learning, computer vision, AI for health, and human visual perception.



Sheng Wen (Senior Member, IEEE) received the Ph.D. degree from Deakin University, Australia, in 2014. He is currently working full-time as an Associate Professor with Swinburne University of Technology, Australia. He is also the Director of Blockchain Innovation Lab and the Deputy Director of Swinburne Cybersecurity Lab at Swinburne University, Australia. In addition, he has published over 100 high-quality papers, including top conference papers such as papers in IEEE Symposium on Security and Privacy (S&P), ACM Conference on Computer and Communications Security (CCS), USENIX Security Symposium (USENIX Security), Network and Distributed System Security Symposium (NDSS), International Conference on Software Engineering (ICSE), Fast Software Encryption (FSE), and The Web Conference (WWW), as well as many papers in IEEE/ACM transactions series journals. Some of these papers have high impact factors or become highly cited papers. His research interests include social network analysis and system security.



Yang Xiang (Fellow, IEEE) received his Ph.D. in computer science from Deakin University, Australia. He is currently a full professor and the Director of the Digital Capability Research Platform, Swinburne University of Technology, Australia. His research interests include the broad area of cybersecurity and AI, which covers the security and algorithms of software, systems, networks, and applications to solve the security, privacy, reliability, responsiveness, and robustness challenges. He is also leading the Blockchain initiatives at Swinburne. In the past 20 years, he has published more than 300 research papers in top-tier international journals and conferences. He is the Editor-in-Chief of the *SpringerBriefs on Information Security and Cryptography*. He is serving as the Associate Editor of the *ACM Computing Surveys*. He is a current member of College of Experts (CoE) of the Australian Research Council (ARC). He is a Fellow of the IEEE.